Does Adaptive Micro-Batch Triggering Help Spark Structured Streaming?

A Fault-Injection Study of Replay, Recovery and Latency

Rajshekar Medipally
Independent researcher
https://github.com/rmedipallycic/spark-streaming-fault-tolerance

October 2026

Abstract

Checkpoint-interval tuning is usually framed as a trade-off between overhead in normal operation and work lost on failure, and adaptive controllers aim to checkpoint more often when failures look likely. Spark Structured Streaming has no separate checkpoint interval: its processing-time trigger schedules micro-batches, and the offset and commit logs that recovery relies on are written once per batch. We therefore studied adaptive *trigger* control and asked whether the classic framing carries over. We built a fault-injection harness that SIGKILLs a Spark job reading sequence-numbered records from Kafka, restarts it from its checkpoint, and counts every duplicated or missing record, and implemented a controller (ACS) that changes the trigger at runtime by restarting the query between batches. Over 210 measured trials on one machine with a light, low-rate workload, a crash caused a batch to be reprocessed in 9 of 120 comparison trials, because records that arrive between batches are read for the first time after a restart, not replayed; when it did happen, the replay was the interrupted batch and reached 15,750 records. Recovery took about 10 s, with no systematic relationship to the interval, and was dominated by restart. In this setting the interval traded end-to-end latency against batch-commit count rather than against lost work. Interval switches were cheap (median 0.54 s), and no duplicates were observed in the 320 tested switches. On pairs with matched workload seeds and kill delays, a fixed interval set to ACS's own mean matched or beat it: against a fixed 5.8 s interval, the original ACS rule had the lower 95th-percentile latency in 0/10 pairs and fewer commits in 2/10. Failure-risk-driven adaptation thus showed no measurable benefit under the tested workload; we do not claim the same for heavily loaded or stateful jobs. We also report two measurement pitfalls: an earlier simulation that predicted the wrong failure behaviour, and laptop standby that silently paused six trials, which we detected from the producer's clock.

1 Introduction

Fault-tolerant stream processors periodically record how far they have read and what state they hold, so that after a failure they can resume instead of starting over. How often to do this is a classic question [12, 5]: checkpoint too often and normal operation pays for it; too rarely and a failure throws away more work. Adaptive policies that tighten the interval when the system looks unstable follow naturally from that framing.

We set out to build and evaluate such a policy for Spark Structured Streaming [1]. An earlier simulation of the policy, Adaptive Checkpoint Selection (ACS), suggested large reductions in duplicated records. We keep the name for continuity, but what ACS controls in Spark is the processing-time trigger of the micro-batches, not an independent checkpoint or state-snapshot interval: offset and commit metadata are persisted once per batch, so the trigger sets their frequency. Our study is therefore one of adaptive micro-batch trigger control, and it does not address asynchronous checkpointing or state-store snapshot policies. When we audited that simulation, we found that its advantage followed from how the model was written, and we replaced it with live measurements on real Spark. This paper reports what those measurements showed.

The answer was negative for the workload we tested, for a reason that is specific to micro-batch execution and easy to miss. In Structured Streaming the processing-time trigger sets how often a batch runs, and with it how often offset and commit entries are written. Records that arrive between two triggers are not lost or replayed when the driver dies: they are simply read for the first time by the restarted query. Only a batch that is *running* at the moment of the crash is redone. Longer intervals therefore do not mean more lost work in the usual sense; they mean higher latency, because records wait longer for the next batch. In our setting the interval acted as a knob between latency and batch-commit count, and a well-chosen fixed value served that trade-off as well as adaptive control did.

Contributions.

1. A fault-injection harness for Spark Structured Streaming with Kafka that counts every duplicated and missing record by sequence number, measures replay from the checkpoint logs, end-to-end latency per record, and recovery time, and records the code commit and environment of every trial.

2. A measured account of delivery under driver failure and checkpoint corruption for the exactly-once file sink and an at-least-once foreachBatch sink (84 trials).

3. A live, restart-based implementation of adaptive micro-batch trigger control, with its switching cost, and an analysis showing that the original ACS rule cannot react to any single signal.

4. A paired comparison of ACS against fixed intervals, including fixed intervals set to ACS's own mean, in two runs (60 and 66 trials), with paired differences and confidence intervals, showing no measurable advantage for adaptation under the tested workload, and a simple model of why.

5. Two methodology lessons: simulated failure behaviour that differed from the measured behaviour in kind, and a clock-based test that detected trials paused by laptop standby.

2 Background

Micro-batch execution. Structured Streaming runs a streaming query as a sequence of small batch jobs [1, 13]. With a processing-time trigger of T seconds, the driver starts a batch every T seconds if new data is available. Before a batch runs, the driver writes the range of source offsets it will read to the checkpoint's `offsets/` log; after the sink has finished, it writes an entry to `commits/`. On restart, Spark finds the latest offsets entry; if it has no matching commit, the batch is run again with the same offsets.

End-to-end guarantees. Exactly-once output needs a replayable source (Kafka offsets [8]), deterministic re-execution of a batch from its recorded offsets, and an idempotent or transactional sink [10]. Spark's file sink provides the last part through its own `_spark_metadata` log, which lists the files that belong to each committed batch. A `foreachBatch` sink hands each batch to user code; if that code is not idempotent, a batch that is run again is written again (at-least-once).

What the trigger controls. The trigger schedules micro-batches; it is not a checkpoint interval in the sense of an independently configurable periodic snapshot. Because offset and commit entries are written once per batch, however, the trigger determines how often they are written, and that is the quantity this paper varies. (Stateful queries also write state-store files per batch, with snapshot frequency set separately; our queries are stateless.) Section 7 shows why, unlike in the classic model, a longer interval does not proportionally increase the work redone after a crash.

3 Harness and setup

One trial. A trial creates a fresh Kafka topic and produces 36,000 records with sequence numbers through an idempotent producer, so that any duplicate is Spark's. It starts the Spark job from the earliest offset, waits for the first committed output, injects the failure 15 s plus a per-trial phase later, restarts the job from the same checkpoint, waits until output stops changing, and stops the query gracefully. It then counts duplicated and missing sequence numbers in the committed output and, separately, in every output file. Each trial writes a JSON record with its configuration, git commit (and whether the working tree was dirty; 0 trials were run from a dirty tree), package versions, event timestamps and counts, plus the Spark log, the controller log and a checkpoint snapshot taken before the restart.

Failures. *Driver kill:* SIGKILL of the Spark process group (in local mode, driver and executors are one JVM), then restart. *Offset-log corruption:* the same kill, then random bytes written over the middle of the latest file in `offsets/`, then restart.

Metrics. *Recovery:* time from the kill to the first new entry in `commits/` after the restart. *Replay:* records in batches that had an offsets entry but no commit at the kill, read from the checkpoint before the restart. *End-to-end latency:* per record, the modification time of the first committed output file containing it minus the producer's send timestamp, which the producer writes into the record; we report each trial's median (p50) and 95th percentile (p95). This approximates end-to-end latency: a file's modification time marks when its last write finished, which can differ slightly from when a downstream reader would see it. Producer, Spark and file system run on one machine, so both timestamps come from the same clock. A record that was written again after a replay counts once, at its earliest file. *Batch-commit count (overhead proxy):* the number of committed micro-batches in the trial (every trial processes the same 36,000 records). Each batch writes one offsets and one commits entry and pays Spark's per-batch scheduling cost, so the count tracks this overhead; we did not measure checkpoint bytes, I/O time or CPU,

and fewer commits need not mean proportionally lower resource use.

Environment. One laptop (Intel Core i5-1335U) running WSL2 Ubuntu with 8 GB for WSL; Spark Structured Streaming 3.5.3 through PySpark in `local[2]` mode; Java 17; one Kafka broker (Confluent 7.5.0) in Docker Compose, with a 4-partition topic per trial. Everything ran on the same machine, so network and cluster scheduling play no part; Section 9 discusses what this does and does not allow us to conclude.

4 Study 1: what Spark delivers under failure

The first study used two fixed triggers, A (1 s) and B (30 s), both sinks, both failures, and 10 trials per cell plus one failure-free baseline each, at a constant 300 records/s (84 trials, code commits 4a2dfe3, 5607d08; an earlier attempt at B, invalidated by a full `/tmp`, is kept and documented in the repository). Kill times were spread across the trigger interval by a golden-ratio sequence.

Table 1 shows three results. The file sink delivered every record exactly once in all 20 driver kills; even the uncommitted files contained no duplicates. The at-least-once foreachBatch sink duplicated records in one trial: 300 records, exactly one 1-second batch at 300 records/s, written again after the restart. A duplicate requires the kill to land between a batch's write and its commit, a short window that none of the ten 30 s kills hit. Offset-log corruption stopped the pipeline in 40 of 40 trials: Spark refused to restart (`MalformedInputException`, exit code 3) instead of continuing with wrong offsets, so no records were duplicated or silently lost; processing simply stopped until an operator intervened.

Recovery after a driver kill took 9.5–28.6 s. From the timestamps in each Spark log, a median 67% (at least 57%, over 40 kills) of that time passed before the restarted query was running, i.e. in JVM and Spark start-up.

Simulation versus measurement. Table 2 compares these measurements with the earlier simulation of the same strategies. The simulation assigned every failure a constant duplicate rate, including 0.18% (A) and 2.96% (B) for checkpoint corruption, where the measured pipeline instead stopped. It had no model of a sink, so it could not represent the file sink, which admits no duplicates by design, and it predicted recovery growing about fivefold from A to B. Measured, B recovered a few seconds slower than A in this study, but A and B ran on different days; when every strategy was interleaved (Section 6), we observed no systematic relationship between recovery time and interval. The differences are in kind, not degree, and they are why this paper reports only measured results.

5 Adaptive trigger control (ACS), live

The rule. ACS evaluates every 5 s a risk score $r = 0.4\,u + 0.3\,e + 0.3\,f$ from three signals in $[0, 1]$: throughput instability u (coefficient of variation of input rate over the last ten windows, saturating at 0.3; 0.2 during warm-up), error pressure e, and failure recency $f = \max(0, 1 - t_{\text{since failure}}/120\,\text{s})$. If $r > 0.45$ the interval shrinks by 30%; otherwise the window counts as stable, and after four stable windows the interval grows by 20%, within 2–45 s, starting at 10 s. In the simulation, e counted exceptions per 1,000 records. A healthy live pipeline has none, so the live controller measures e as the fraction of batches in the window whose run time exceeded the trigger interval, a measurable sign of falling behind.

A structural flaw (v1). Implementing the rule exposed a property the simulation had hidden: no single signal can cross the threshold. Instability alone contributes at most 0.4, and a recent failure or full error pressure at most 0.38 (with warm-up instability). And because every window with $r \le 0.45$ counts as stable, the interval *relaxes* during a burst unless a failure happened within the previous 100 s. We therefore evaluate two variants, both fixed before any measured run: **v1**, the rule exactly as specified, and **v2**, which tightens when $r > 0.30$ (any one saturated signal), counts a window as stable only when $r < 0.15$, and otherwise holds the interval.

Changing the interval of a running query. A running Structured Streaming query cannot change its trigger. Our controller runs inside the Spark job and, to apply a new interval, waits until the query is between batches, stops it, and starts it again on the same checkpoint in the same Spark session, so no JVM restart is involved. Waiting for the gap between batches matters: stopping a query interrupts a running batch, and for foreachBatch a batch that had already written its output would be written again. The controller persists its interval and a clean-stop flag next to the checkpoint, so that after a crash the restarted job resumes the interval and counts the crash for the recency signal.

Switching cost. Over 320 switches in 40 ACS trials, the time from stopping the old query to the first batch of the new one had a median of 0.54 s (0.52–2.07 s; 33

Table 1: Study 1: driver kill and offset-log corruption (10 trials per cell; baselines: 1). Missing records are counted only for trials whose restart succeeded. Recovery: median (min–max), seconds.

Failure	Sink	Trigger	Trials	Outcomes	Trials w/ dup.	Max dup. rows	Missing	Recovery s
driver kill	file	A (1 s)	10	ok 10	0	0	0/10	11.0 (9.5–15.5)
driver kill	file	B (30 s)	10	ok 10	0	0	0/10	14.8 (12.0–17.0)
driver kill	foreachBatch	A (1 s)	10	ok 10	1	300	0/10	10.0 (9.5–28.6)
driver kill	foreachBatch	B (30 s)	10	ok 10	0	0	0/10	13.3 (11.0–20.5)
offset-log corruption	file	A (1 s)	10	restart failed 10	0	0	0/0	–
offset-log corruption	file	B (30 s)	10	restart failed 10	0	0	0/0	–
offset-log corruption	foreachBatch	A (1 s)	10	restart failed 10	0	0	0/0	–
offset-log corruption	foreachBatch	B (30 s)	10	restart failed 10	0	0	0/0	–
none (baseline)	file	A (1 s)	1	ok 1	0	0	0/1	–
none (baseline)	file	B (30 s)	1	ok 1	0	0	0/1	–
none (baseline)	foreachBatch	A (1 s)	1	ok 1	0	0	0/1	–
none (baseline)	foreachBatch	B (30 s)	1	ok 1	0	0	0/1	–

Table 2: Driver failure: earlier simulation versus Study 1. Duplicate rate: duplicated rows as a share of records, mean over trials; fBatch: foreachBatch sink. Recovery: simulated mean versus measured median, both sinks.

	Simulated		Measured		
Trigger	dup. %	rec. s	file	fBatch	rec. s
A (1 s)	0.19	5.1	0.00	0.083	10.5
B (30 s)	4.43	25.3	0.00	0.000	14.3

of 318 measured switches took more than 1 s), against about 10 s for crash recovery. The other 2 switches have no timing because the injected driver kill came within 0.1 s after them, before the new query had run a batch; those trials then recovered like any other. No ACS trial produced a duplicate (0 trials with duplicates), so no duplicates were observed in the tested switches. This is an observation for our sink, workload and stop procedure, not a proof that restart-based switching is safe in general; it relies on stopping only between batches. Switching was cheap here; whether it is *useful* is the question of the next two sections.

6 Study 2: ACS against fixed intervals

Design. Six strategies: fixed triggers of 1 s (A) and 30 s (B), ACS v1 and v2, and two further fixed triggers set to ACS's mean intervals from the first run, 3.3 s and 5.8 s (C). The C strategies are the fair comparison: they answer whether *adapting* helps, rather than merely choosing a value between 1 s and 30 s. The workload was bursty: every 15 s the producer's rate was drawn from 0.5×, 1× and 3× 300 records/s, from a seed equal to the trial number. All trials used the foreachBatch sink and a driver kill at 15 s plus a phase spread over 30 s, the same for every strategy. Trial t therefore gave every strategy a matched workload seed and a matched kill delay, so strategies can be compared pair by pair. The match is in the schedule, not in the exact failure point. The producer and the Spark job start together; the kill delay is counted from the first committed output, which came 11 s (9.5–21.6 s) after the producer started, mostly Spark start-up, since the first batch runs as soon as the query starts whatever the trigger. Within a trial number, the kill therefore fell at slightly different times relative to the start of production (spread across strategies: median 2.3 s, at most 12.1 s, second run), and at different points of each strategy's own batch cycle. The pairs share input and failure phase, not an identical system state at the kill.

Two runs. The first run (60 trials) ran A, B and both ACS variants interleaved overnight, and the two C strategies the next morning, separately from the others. The 5.8 s C trials recovered more slowly (median 14.3 s against about 10 s), and their Spark start-up was slower too, which no trigger setting can cause, so the first run's comparison with C is biased against C. The second run (66 trials) interleaved all six strategies trial by trial and is our main evidence. Six of its trials were paused by laptop standby (Section 8); they are kept in the data, excluded from the tables, and were rerun with the same trial numbers, giving 60 clean trials, 10 per strategy. Code commits: 369028b, 3d1f93f, 7d66520.

Results. Table 3 and Figure 1 show the second run. The fixed intervals trace a clean trade-off: from A to B, median latency rises from 0.84 s to 13.98 s while batch commits fall from 82 to 6. The question is where ACS lies relative to that curve.

ACS v1 (mean interval 5.8 s) had a median latency of 3.68 s with 24 commits; the fixed curve at the same number of commits gives 2.73 s. Against the fixed 5.8 s interval, it had a higher p95 latency (10.88 versus 8.11 s) and more commits (24 versus 18). Pair by pair (Table 5), v1 had the lower median latency in 5/10 pairs, the lower p95 in 0/10, and fewer commits in 2/10.

Table 3: Study 2, second (interleaved) run: per-trial medians, then median (min–max) over 10 trials. Latency: producer send to output. Batch commits: committed micro-batches per trial (36,000 records each), an overhead proxy. Mean interval: time-weighted, ACS only.

Strategy	Trials	Latency p50 s	Latency p95 s	Batch commits	Mean interval s	Switches	Recovery s
A (1 s)	10	0.84 (0.71–1.38)	8.75 (3.79–14.39)	82 (43–101)	–	–	10.5 (9.6–16.6)
C (3.3 s)	10	2.18 (1.91–2.91)	8.61 (4.29–10.83)	28 (17–40)	–	–	10.5 (8.5–14.1)
ACS v2	10	1.99 (1.43–4.36)	8.89 (6.68–12.15)	35 (14–56)	3.5 (2.9–4.3)	6 (5–6)	10.3 (9.5–14.7)
C (5.8 s)	10	3.46 (3.25–4.27)	8.11 (5.97–11.61)	18 (10–24)	–	–	10.0 (9.5–15.0)
ACS v1	10	3.68 (1.81–6.40)	10.88 (9.29–13.30)	24 (8–34)	5.8 (3.1–6.4)	10 (8–13)	10.3 (9.5–14.1)
B (30 s)	10	13.98 (9.16–24.36)	30.44 (25.76–46.45)	6 (3–6)	–	–	10.5 (10.0–25.6)

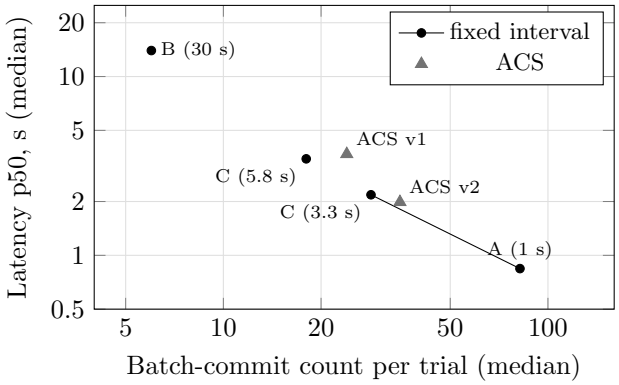


Figure 1: Latency against batch-commit count (overhead proxy), second run. The line joins the fixed intervals (B, C 5.8 s, C 3.3 s, A from left to right). ACS v2 lies on the line; ACS v1 lies above it.

ACS v2 (mean interval 3.5 s) had a median latency of 1.99 s with 35 commits, essentially on the fixed curve (2.02 s at that number of commits). Against the fixed 3.3 s interval it had a slightly lower median latency (1.99 versus 2.18 s; lower in 7/10 pairs), a higher p95 (8.89 versus 8.61 s; lower in only 1/10) and more commits (35 versus 28; fewer in only 1/10).

The first run (Table 6) shows the same ordering for A, B and the two ACS variants. Its C strategies ran separately, and the 5.8 s one had higher latency than in the second run, which made ACS v1 look better against it (9/10 pairs with lower median latency); even so, every ACS variant used more commits than the fixed interval at its own mean in both runs.

Paired differences. Counting wins does not show how large a difference is, or whether a small one could be hidden. Table 4 gives, for each comparison, the mean of the ten per-pair differences (ACS minus C) with a t-based 95% confidence interval. In the second run, both variants had a higher p95 latency than the fixed interval at their mean (v2: +1.14 [+0.09, +2.20] s; v1: +2.71 [+1.43, +3.99] s) and more commits (v2: +6.7 [+3.0, +10.4]; v1: +5.2 [+1.4, +9.0]), with intervals that exclude zero. For median latency the intervals include zero (v2: −0.02 [−0.47, +0.44] s; v1: +0.04 [−0.80, +0.88] s). That does not make the strategies equivalent: with ten pairs, a median-latency gain or loss of up to about half a second (v2) or nearly a second (v1) cannot be ruled out. The first run's intervals favour ACS on median latency, but its C trials ran under different conditions (above). The intervals assume roughly normal per-pair differences, and we did not correct for the twelve comparisons; we read them as a description of effect sizes, not as significance tests.

7 Why adaptation showed no benefit here

Replay was rare. In the classic model, a failure loses the work done since the last checkpoint, so a longer interval means more redone work. In micro-batch Spark, records that arrived since the last batch were never processed; after a restart they are read for the first time. Only a batch that was running when the driver died is run again. Across both runs, that happened in 9 of 120 clean trials (Table 7).

When it happens, it scales with the batch. The replayed amount is the size of the interrupted batch, so it grows with the interval and the current rate. Replay is therefore rare here, not small: the largest, 15,750 records, came from a single 30 s batch. Our ten trials per strategy are too few to estimate expected replay precisely, so we report the events individually and give a simple model of why it need not grow in proportion to the interval.

An illustrative model. Let T be the trigger interval, D the run time of a batch, λ the input rate and R the number of records replayed after a crash. Consider only the non-overloaded regime $D < T$, in which each batch finishes before the next trigger and no backlog builds up, and assume a roughly stable input rate, batches that start every T seconds, and a crash time uniformly

Table 4: Paired differences, ACS minus the fixed interval at its mean: mean over the 10 pairs, with t-based 95% confidence interval in brackets. Negative favours ACS. Each entry is an interval for the *mean paired difference* of per-trial values; it is not an interval for the medians over trials reported in Table 3, which are a different statistic. Latency in seconds; commits per trial. Run 1's C trials ran separately from the others (see text).

Run	Comparison	Δ latency p50	Δ latency p95	Δ batch commits
1	ACS v2 − C (3.3 s)	−0.27 [−0.52, −0.02]	+0.90 [−0.58, +2.37]	+8.2 [+4.9, +11.5]
1	ACS v1 − C (5.8 s)	−0.68 [−1.35, −0.01]	−1.48 [−3.13, +0.17]	+6.9 [+2.2, +11.6]
2	ACS v2 − C (3.3 s)	−0.02 [−0.47, +0.44]	+1.14 [+0.09, +2.20]	+6.7 [+3.0, +10.4]
2	ACS v1 − C (5.8 s)	+0.04 [−0.80, +0.88]	+2.71 [+1.43, +3.99]	+5.2 [+1.4, +9.0]

Table 5: Paired comparison: in how many of the 10 pairs (matched workload seed and kill delay) ACS had the lower value.

Run	Comparison	p50	p95	Commits
1	ACS v2 vs C (3.3 s)	6/10	3/10	0/10
1	ACS v1 vs C (5.8 s)	9/10	7/10	2/10
2	ACS v2 vs C (3.3 s)	7/10	1/10	1/10
2	ACS v1 vs C (5.8 s)	5/10	0/10	2/10

Table 6: Both runs, medians over clean trials. Run 1's C strategies ran separately from the others (see text).

	Run 1			Run 2		
Strategy	p50	p95	comm.	p50	p95	comm.
A (1 s)	0.76	6.10	84	0.84	8.75	82
C (3.3 s)	2.12	7.01	30	2.18	8.61	28
ACS v2	1.73	8.36	37	1.99	8.89	35
C (5.8 s)	3.96	11.92	17	3.46	8.11	18
ACS v1	3.48	10.27	26	3.68	10.88	24
B (30 s)	12.65	29.23	6	13.98	30.44	6

distributed over the batch cycle. Then

$$P = P(\text{crash during a batch}) \approx D/T,$$
$$E[R] \approx P \cdot E[B \mid \text{interrupted}],$$

where B is the interrupted batch's record count. A batch holds what arrived during the previous cycle, $B \approx \lambda T$, so $E[R] \approx \lambda D$: to first order the expected replay is set by how long a batch runs, not by the interval, whereas the classic model gives expected lost work of about $\lambda T/2$. The interval enters only through D. Writing $D \approx d_0 + cB$, with a fixed per-batch cost d_0 and a per-record cost c, gives $E[R] \approx \lambda(d_0 + c\lambda T)$, which grows with T only in proportion to $c\lambda^2$. For our light transformation and low rate that slope is small; for heavy or stateful batches, or higher rates, it need not be, and replay could grow with the interval. The approximation $B \approx \lambda T$ also weakens under bursts: interrupted batches are then larger than average, because large batches run longer and are more likely to be hit. The overloaded regime $D \geq T$ is different and we do not model it: Spark starts the next batch as soon as the previous one ends, a crash almost always hits a running batch, and the batch size depends on how the backlog evolves rather than on T. The model is an explanation of the mechanism, not a validated prediction; we did not fit it to the data.

Table 7: Every trial in which the kill interrupted a running batch (replay > 0), both runs. Dup.: duplicated rows in the output.

Run	Strategy	Trial	Replayed	Recovery s	Dup.
1	ACS v2	2	89	9.5	0
1	A (1 s)	3	150	9.0	0
1	A (1 s)	8	150	9.0	0
1	ACS v2	4	236	9.5	0
1	ACS v2	9	490	10.0	0
1	C (5.8 s)	7	1,739	14.0	1,739
2	A (1 s)	7	301	10.5	0
2	ACS v2	7	981	10.0	0
2	B (30 s)	1	15,750	10.5	0

Replay did not lengthen recovery, and rarely duplicated. Recovery in the replay trials was 9.0–14.0 s, within the range of all other trials; the 15,750-record replay recovered in 10.5 s. Only one of the nine replays produced duplicates (1,739 rows): the interrupted batch had already written its output, the at-least-once window from Study 1. In the others the batch died before its write and was simply redone.

No systematic relationship between recovery and interval. Median recovery was 10.0–10.5 s for every strategy in the second run (Table 3). Splitting each recovery at the moment the restarted query was running (Table 8), start-up took about 7 s and the first batch after it about 3.5 s, for every strategy. Over both runs, 90 of 120 recoveries took under 12 s and 29 took 12–17 s; the slower ones had both slower start-up (median 9.1 s against 6.7 s) and a slower first batch (5.0 against 3.3 s), which points to the machine's state at the time rather than to the strategy. One recovery took 25.6 s, with an 18 s start-up.

Table 8: Recovery split, second run: Spark start-up after the restart, and the first batch after start-up. Median (min–max), seconds.

Strategy	n	Start-up	First batch
A (1 s)	10	7.2 (6.5–10.7)	3.5 (2.7–5.8)
C (3.3 s)	10	7.2 (6.1–8.9)	3.3 (2.4–6.3)
ACS v2	10	7.0 (6.2–11.0)	3.5 (2.7–4.3)
C (5.8 s)	10	6.7 (6.3–9.8)	3.4 (2.8–5.2)
ACS v1	10	7.2 (6.3–9.9)	3.4 (2.9–5.0)
B (30 s)	10	7.0 (6.5–18.0)	3.6 (3.0–7.5)

Table 9: Stalled trials (producer more than 10 s behind schedule), excluded from the tables and rerun with the same trial numbers.

Run id (start, UTC)	Strategy	Trial	Overrun s
20261016T223348Z	C (3.3 s)	0	2431
20261017T000019Z	B (30 s)	2	575
20261017T003005Z	C (3.3 s)	3	376
20261017T005701Z	A (1 s)	4	264
20261017T010347Z	B (30 s)	4	684
20261017T012115Z	C (5.8 s)	4	1691

What is left is latency against batch commits. With replay rare and no systematic relationship between recovery and interval in these runs, the interval governed how long records wait for a batch (latency) and how many batches commit (the overhead proxy). That trade-off depends on the workload's rate and the latency target, both of which an operator knows in advance. A fixed interval chosen for the target serves it directly. ACS adds switching (cheap, but not free), and its decisions follow failure recency and throughput variance, which are signals for the risk of losing work, the cost that was small under the tested workload.

8 Detecting paused trials

Six trials of the second run took minutes longer than any other, with p95 latencies up to 536 s. Their outcomes were correct; the timings were not Spark's. The producer is paced only by the wall clock, so the time it needs to send all records follows from the workload schedule. We define a trial as *stalled* if its producer finished more than 10 s later than its schedule allows. Over 126 trials, 120 finished within 1 s of schedule, none between 1 and 10 s, and 6 were 4 to 40 minutes late (Table 9). The Windows power log showed that the laptop had entered Modern Standby after an idle timeout during each of those periods, although it was plugged in; an excerpt is in the repository. The test needs no access to the operating system and catches a pause wherever it falls in a trial; checking only the time to first output would have missed five of the six.

9 Threats to validity

- **One machine, local mode.** Driver and executors share one JVM, so executor-only failures, network partitions and multi-node scheduling are not covered. On a cluster, batches take longer to schedule and the in-flight window is larger, which could make replay more frequent; the mechanism in Section 7 still applies, but its magnitude may differ.
- **One workload, low rate, stateless.** 300 records/s with 0.5–3× bursts and a trivial stateless transformation. Heavier or stateful batches spend more time in flight, which raises both the chance that a kill interrupts one and, by the model in Section 7, how fast replay grows with the interval. Our conclusions are limited to this workload.
- **Overhead proxy.** We count committed batches; we did not measure checkpoint metadata bytes, I/O time or CPU.
- **Ten trials per strategy and run.** Enough to show the ordering and its consistency across pairs and runs, not to establish equivalence or to estimate rare-event rates; confidence intervals that include zero (Table 4) leave room for small differences. Replay events are reported individually for that reason.
- **Latency measurement.** File modification time approximates when output became visible; it is precise to the file system's timestamp resolution and needs no clock synchronization only because everything ran on one machine.
- **Machine state.** Recovery showed two modes tied to the machine's state, and the first run's C strategies ran under different conditions. The second run interleaves every strategy trial by trial; its six reruns ran separately and recovered in the slower mode.
- **One adaptive rule.** We tested ACS as specified and one revision. A controller that targets latency directly, or tunes batch size as in [6], is a different question; our results bound what an interval controller driven by failure risk can gain here, not every possible controller.
- **One kind of corruption.** Overwriting part of the latest offsets file; other corruptions (commit log, sink metadata, truncation) may behave differently.

10 Related work

Discretized streams [13] introduced micro-batch stream processing with lineage-based recovery, and Structured Streaming [1] its declarative successor with exactly-once sinks. Das et al. [6] adapt the batch interval of Spark Streaming to keep the system stable as load changes, a latency and stability goal rather than a fault-tolerance one, consistent with our finding that the interval is a latency knob. Drizzle [11] decouples the processing interval from coordination to cut micro-batch overhead. Flink takes consistent snapshots with barriers derived from Chandy–Lamport [4, 3, 2], where the checkpoint interval and the processing granularity are separate and the classic trade-off applies more directly. Optimal checkpoint intervals under failure were derived by Young [12] and Daly [5], and rollback recovery is surveyed by Elnozahy et al. [7]. The same measurement approach, a fault-injection harness with committed raw logs, was applied to LLM serving in [9].

11 Conclusion

In Spark Structured Streaming on one machine, with a light, low-rate workload, a driver crash rarely forced a batch to be redone (9 of 120 trials), because records that arrive between batches are read after the restart, not replayed; when a batch was redone, it could be large (up to 15,750 records). Recovery took about 10 s, mostly in restart, and we observed no systematic relationship between recovery time and trigger interval. The interval therefore traded latency against batch-commit count. Changing it at runtime was cheap, and no duplicates were observed in the tested switches, but on pairs with matched workloads and kill delays a fixed interval at ACS's own mean matched or beat it, and the original ACS rule could not react to any single signal. Failure-risk-driven adaptation of the trigger showed no measurable benefit under the tested workload; for such workloads our evidence supports choosing a fixed trigger for the latency target. Whether this holds for heavily loaded or stateful jobs is open. Next steps: a multi-node cluster with executor failures and heavier batches, where in-flight batches are larger; direct measurement of checkpoint I/O; other corruption modes; and controllers that target latency directly.

Artifact. Code, every trial record and log, and the script that generates every table, figure and in-text number of this paper are at `https://github.com/rmedipallycic/spark-streaming-fault-tolerance` (MIT licence). Excluded trials are kept with their reasons. Appendix A lists how to reproduce the tables and the runs.

Acknowledgments

The harness code, analysis scripts, and much of the text of this paper were developed with substantial help from an AI assistant (Claude, Anthropic). The author directed the work, ran all experiments, and is responsible for every reported result; each number is generated by `paper/make_tables.py` from the trial records committed to the repository.

A Reproducibility checklist

Regenerating the paper's numbers. From the repository root, `python paper/make_tables.py` (Python 3, no packages beyond the standard library) reads the committed trial records and rewrites every file in `paper/tables/`; `pdflatex main`, `bibtex main` and two more `pdflatex main` runs in `paper/` rebuild the PDF. The expected output is byte-identical tables; `git status` after running the script should show no changes in `paper/tables/`. Per-trial CSV and Markdown summaries are produced by `python live/analyze.py` (Study 1) and `python live/analyze_acs.py --root experiments/live_acs2` (and `live_acs`).

Software. WSL2 Ubuntu; OpenJDK 17; Python 3.11; PySpark 3.5.3; confluent-kafka 2.3.0; Docker Compose with the Confluent 7.5.0 ZooKeeper and Kafka images (`docker-compose.yml`). `bash live/setup_wsl.sh` installs these. Every `trial.json` records the exact versions, command line, git commit and whether the tree was dirty.

Running the trials. With Kafka up (`docker compose up -d zookeeper kafka`): Study 1, `TRIALS=10 bash live/run_grid.sh`; Study 2, first run, `bash live/run_acs_grid.sh` and then `STRATEGIES="C3.3 C5.8" bash live/run_acs_grid.sh`; second run, `STRATEGIES="A B C3.3 C5.8 ACSv1 ACSv2" EXPERIMENT=live_acs2 bash live/run_acs_grid.sh`. The scripts refuse to start if `live/` or `src/` has uncommitted changes.

Seeds and kill times. In Study 2, trial t uses workload seed t (15 s segments at 0.5, 1 or 3× 300 records/s) and a kill $15 + 30\,\{0.618t\}$ seconds after the first committed output, where $\{\cdot\}$ is the fractional part; every strategy therefore gets the same input schedule and kill delay; the delay is counted from the first committed output, so the wall-clock kill time differs slightly between strategies (Section 6). Study 1 uses the same golden-ratio phase, scaled by the trigger interval.

Exclusion rules. A trial is *stalled*, and left out of the tables but kept in the data, if its producer finished more

than 10 s behind its rate schedule (`producer_overrun` in `live/analyze_acs.py`); Table 9 lists the six. Stalled trials were rerun with the same trial number. Only trials with `run_type measured` are used; smoke tests and the invalidated first attempt at B in Study 1 (full `/tmp`) are kept separately and documented in the repository.

Expected outputs. 210 measured trials; 120 clean comparison trials, 10 per strategy and run; 9 replay events; 320 switches, of which 318 timed; no duplicates in any ACS trial. Absolute latencies and recovery times depend on the machine; the ordering of strategies in Table 3 is the result to compare.

References

[1] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[2] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. State management in Apache Flink: Consistent stateful distributed stream processing. *Proceedings of the VLDB Endowment*, 10(12):1718–1729, 2017.

[3] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows, 2015. arXiv:1506.08603.

[4] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[5] J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

[6] Tathagata Das, Yuan Zhong, Ion Stoica, and Scott Shenker. Adaptive stream processing using dynamic batch sizing. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2014.

[7] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[8] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.

[9] Rajshekar Medipally. What does a streaming client see when an LLM server crashes? a fault-injection study of client-visible token anomalies in vLLM and SGLang, 2026. Preprint, doi:10.5281/zenodo.23377448.

[10] The Apache Software Foundation. Structured streaming programming guide (apache spark 3.5), 2024. `https://spark.apache.org/docs/3.5.3/structured-streaming-programming-guide.html`.

[11] Shivaram Venkataraman, Aurojit Panda, Kay Ousterhout, Michael Armbrust, Ali Ghodsi, Michael J. Franklin, Benjamin Recht, and Ion Stoica. Drizzle: Fast and adaptable stream processing at scale. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.

[12] John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

[13] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013.